

Databricks

Data Engineer Associate

Complete Coding Tasks Reference

All SQL · PySpark · DAB · CLI tasks — organized by the official May 2026 exam syllabus

45 Questions	90 Minutes	\$200 USD	2 Years Valid
Multiple Choice	Time Limit	Registration	Certification

Contents

1 · Databricks Platform (~10%)

- Delta Lake — DESCRIBE commands
- CREATE TABLE variants
- Compute types comparison

2 · Data Ingestion (~20%)

- Auto Loader — basic & schema evolution
- Auto Loader — directory listing vs file notification
- COPY INTO — JSON, CSV
- COPY INTO vs Auto Loader comparison
- Lakeflow Connect — architecture & connectors
- JDBC ingestion in notebooks

3 · Transformation & Modeling (~25%)

- Delta DML — INSERT / UPDATE / DELETE
- MERGE INTO — SQL & PySpark DeltaTable API
- Time Travel — query & RESTORE
- OPTIMIZE, VACUUM, Liquid Clustering
- PySpark — read/write formats
- PySpark — column & row operations
- PySpark — aggregations (count, sum, approx_count_distinct)
- PySpark — all join types + broadcast
- PySpark — deduplication
- PySpark — window functions
- PySpark — UDFs
- Spark SQL — aggregations, HAVING, CASE WHEN
- Spark SQL — window functions
- Spark SQL — higher-order array functions
- Spark SQL — views (persistent, temp, materialized)
- Lakeflow SDP — Bronze / Silver / Gold (new API)
- Lakeflow SDP — CDC with create_auto_cdc_flow
- Spark tuning parameters

4 · Lakeflow Jobs (~18%)

- Job / Task / Trigger — concepts & limits
- Control flow — branching, looping, retries
- DAB YAML — job with if/else branching
- dbutils — widgets, notebook run, fs, secrets

5 · CI/CD (~12%)

- Databricks Repos & Git — CLI commands
- DAB bundle structure — databricks.yml
- Databricks CLI — validate, deploy, run

6 · Monitoring & Tuning (~8%)

- Spark UI bottlenecks — skew, shuffle, spill
- Data skew — salting pattern
- Jobs SDK — run history

7 · Governance & Security (~7%)

- Unity Catalog — managed vs external tables
- GRANT / REVOKE / DENY hierarchy
- Row & Column security — dynamic views

- ABAC — row filter functions
- Delta Sharing — share setup
- Lakehouse Federation — foreign catalog

1

Databricks Platform

~10%

01 Delta Lake — DESCRIBE commands

SQL

Three essential DESCRIBE commands — know which returns what.

SQL

```
-- Column names, types, nullability, comments
DESCRIBE TABLE catalog.sales.orders;

-- File-level metadata: location, numFiles, sizeInBytes, partitions
DESCRIBE DETAIL catalog.sales.orders;

-- Full transaction log: version, timestamp, operation, user, metrics
DESCRIBE HISTORY catalog.sales.orders;

-- Storage properties + partition stats
DESCRIBE EXTENDED catalog.sales.orders;
```

Exam tip: DETAIL = file-level metadata (numFiles, sizeInBytes). HISTORY = operation log (who did what, when, which version). These are separate commands — a common exam trap.

02 CREATE TABLE — all DDL forms

SQL

SQL

```
-- 1. CREATE OR REPLACE – drops & recreates regardless of existence
CREATE OR REPLACE TABLE catalog.sales.orders (
  order_id BIGINT NOT NULL,
  customer STRING,
  amount DOUBLE,
  order_date DATE,
  status STRING
) USING DELTA
PARTITIONED BY (order_date)
COMMENT 'Transactional orders';

-- 2. IF NOT EXISTS – only creates if table is missing
CREATE TABLE IF NOT EXISTS catalog.sales.orders (
  order_id BIGINT, customer STRING, amount DOUBLE
);

-- 3. CTAS – Create Table As Select (copies schema + data)
CREATE OR REPLACE TABLE catalog.silver.big_orders
AS SELECT * FROM catalog.bronze.orders WHERE amount > 1000;
```

03 Compute types comparison

Concept

Type	Use Case	Lifecycle	Cost
All-Purpose Cluster	Interactive dev / notebooks	Always-on / auto-terminate	DBU/hr while running
Job Cluster	Automated pipeline runs	Per run — auto-terminates after	Cheaper DBU rate
SQL Warehouse (Classic/Pro)	SQL analytics, BI tools	Auto-suspend after idle	Per-query DBU
Serverless Compute	Instant-start notebooks & jobs	Fully managed by Databricks	Per-second billing

Exam tip: For production pipelines always prefer Job Cluster over All-Purpose. All-Purpose clusters are for development only. Serverless is the default when UC + Serverless is enabled.

2

Data Ingestion & Loading

~20%

01 Auto Loader — basic ingestion

PySpark

PySpark

```
# cloudFiles source – only processes NEW files since last checkpoint
df_stream = (spark.readStream
    .format('cloudFiles')
    .option('cloudFiles.format', 'json')
    .option('cloudFiles.schemaLocation', '/mnt/checkpoints/orders_schema')
    .load('/mnt/raw/orders/'))

# Write to Unity Catalog Delta table
(df_stream.writeStream
    .format('delta')
    .outputMode('append')
    .option('checkpointLocation', '/mnt/checkpoints/orders')
    .trigger(availableNow=True) # process all available, then stop
    .toTable('catalog.bronze.orders'))
```

Exam tip: `trigger(availableNow=True)` is the modern replacement for deprecated `trigger(once=True)`. It processes all currently available data then stops — ideal for batch-style incremental loads.

02 Auto Loader — schema enforcement & evolution

PySpark

PySpark

```
# Schema ENFORCEMENT (default) – rejects records not matching stored schema
df_strict = (spark.readStream
    .format('cloudFiles')
    .option('cloudFiles.format', 'json')
    .option('cloudFiles.schemaLocation', '/ckpt/schema') # stored schema
    .load('/mnt/raw/orders/'))

# Schema EVOLUTION – merges new source columns into target
df_evolve = (spark.readStream
    .format('cloudFiles')
    .option('cloudFiles.format', 'json')
    .option('cloudFiles.schemaLocation', '/ckpt/schema')
    .option('cloudFiles.schemaEvolutionMode', 'addNewColumns')
    .load('/mnt/raw/orders/'))

(df_evolve.writeStream
    .option('checkpointLocation', '/ckpt/orders')
    .option('mergeSchema', 'true') # allow Delta schema merge
    .trigger(availableNow=True)
    .toTable('catalog.bronze.orders'))
```

03 Auto Loader — directory listing vs file notification

Concept

Mode	How it works	Best for
Directory Listing (default)	Lists all files, tracks processed ones in state store	Smaller volumes, simple setup
File Notification	Uses cloud events (S3 SQS / Azure Event Grid) — only new files pushed	Large buckets with millions of files

PySpark

```
# File notification mode — scalable for large buckets
df = (spark.readStream
      .format('cloudFiles')
      .option('cloudFiles.format', 'json')
      .option('cloudFiles.useNotifications', 'true')
      .option('cloudFiles.schemaLocation', '/ckpt/schema')
      .load('/mnt/raw/large-bucket/'))
```

04 COPY INTO — incremental SQL ingestion

SQL

NEW

COPY INTO tracks loaded files — idempotent and safe to re-run. Target table must already exist.

SQL

```
-- JSON from ADLS/S3/GCS into a Unity Catalog Delta table
COPY INTO catalog.bronze.orders
FROM '/mnt/raw/orders/'
FILEFORMAT = JSON
FORMAT_OPTIONS (
  'inferSchema' = 'true',
  'multiline' = 'false'
)
COPY_OPTIONS (
  'mergeSchema' = 'true' -- allow schema evolution
);

-- CSV with header
COPY INTO catalog.bronze.customers
FROM '/mnt/raw/customers/'
FILEFORMAT = CSV
FORMAT_OPTIONS (
  'header' = 'true',
  'inferSchema' = 'true',
  'delimiter' = ','
);
```

05 COPY INTO vs Auto Loader — decision table

Concept

Feature	COPY INTO	Auto Loader
Interface	SQL command	PySpark Structured Streaming
Scale	Thousands of files	Millions of files
Idempotent	Yes (metadata store)	Yes (checkpoint)
Schema evolution	mergeSchema = true	schemaEvolutionMode = addNewColumns
Streaming	No — batch only	Yes — continuous or batch
Best for	SQL-first, small–medium data	Large-scale, continuous ingestion

06 Lakeflow Connect — connector architecture

Concept

NEW

Lakeflow Connect delivers data into Unity Catalog-governed Delta tables, powered by serverless compute and Lakeflow Spark Declarative Pipelines. SaaS connectors use: Connection + Ingestion Pipeline + Destination Tables. Database connectors also need: Ingestion Gateway (runs in your VPC) + Staging Storage.

Connector Type	Examples	Architecture
SaaS connectors	Salesforce, HubSpot, Workday, ServiceNow, Jira	Connection + Pipeline + Destination tables
Database connectors (CDC)	MySQL, PostgreSQL, SQL Server	Gateway (in your VPC) + Staging + Pipeline
Community connectors	Open-source, custom-built	Flexible, community maintained

Exam tip: Ingestion method decision: Auto Loader = files in object storage. COPY INTO = SQL-first small batches. Lakeflow Connect = enterprise SaaS/DB. JDBC notebook = custom one-off with full control.

07 JDBC ingestion in notebooks

PySpark

```
PySpark

# JDBC ingestion – read from external database
jdbc_url = 'jdbc:postgresql://host:5432/mydb'

df = (spark.read
      .format('jdbc')
      .option('url', jdbc_url)
      .option('dbtable', 'public.orders')
      .option('user', dbutils.secrets.get('scope', 'db-user'))
      .option('password', dbutils.secrets.get('scope', 'db-pass'))
      .option('numPartitions', '8') # parallel reads
      .option('partitionColumn', 'order_id') # column to split on
      .option('lowerBound', '1')
      .option('upperBound', '1000000')
      .load())

# Write to Unity Catalog Delta table
(df.write
 .format('delta')
 .mode('overwrite')
 .saveAsTable('catalog.bronze.orders_jdbc'))
```

Exam tip: Never hardcode credentials. Always use `dbutils.secrets.get('scope', 'key')` backed by Azure Key Vault or Databricks-managed secrets.

3

Data Transformation & Modeling

~25%

01 Delta DML — INSERT, UPDATE, DELETE

SQL

SQL

```
-- INSERT INTO — append rows
INSERT INTO catalog.sales.orders
VALUES (1001, 'Alice', 250.0, '2024-01-15', 'confirmed');

-- INSERT OVERWRITE — replaces all data (not append)
INSERT OVERWRITE catalog.sales.orders
SELECT * FROM staging.orders;

-- UPDATE — modify matching rows
UPDATE catalog.sales.orders
SET status = 'cancelled'
WHERE order_date < '2023-01-01' AND status = 'pending';

-- DELETE — remove rows
DELETE FROM catalog.sales.orders
WHERE status = 'test';
```

02 MERGE INTO — full upsert (SQL + PySpark API)

SQL

PySpark

SQL

```
-- SQL MERGE INTO
MERGE INTO catalog.silver.orders t
USING catalog.bronze.orders_updates u
ON t.order_id = u.order_id
WHEN MATCHED AND u.status = 'deleted'
THEN DELETE
WHEN MATCHED
THEN UPDATE SET t.status = u.status, t.amount = u.amount
WHEN NOT MATCHED BY TARGET
THEN INSERT (order_id, customer, amount, order_date, status)
VALUES (u.order_id, u.customer, u.amount, u.order_date, u.status);
```

PySpark

```
# PySpark DeltaTable API
from delta.tables import DeltaTable

dt = DeltaTable.forName(spark, 'catalog.silver.orders')

(dt.alias('t')
 .merge(updates_df.alias('u'), 't.order_id = u.order_id')
 .whenMatchedDelete(condition="u.status = 'deleted'")
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute())
```

03 Time Travel — query & RESTORE

SQL

SQL

```
-- Query a specific past version
SELECT * FROM catalog.sales.orders VERSION AS OF 3;
SELECT * FROM catalog.sales.orders TIMESTAMP AS OF '2024-03-01';

-- Restore table to previous version (in-place rollback)
RESTORE TABLE catalog.sales.orders TO VERSION AS OF 3;
RESTORE TABLE catalog.sales.orders TO TIMESTAMP AS OF '2024-03-01';
```

Exam tip: Time travel only works within the VACUUM retention window (default 7 days). After VACUUM removes old files, those versions are gone. RESTORE writes a new version in the transaction log.

04 OPTIMIZE, VACUUM, Liquid Clustering

SQL

NEW

SQL

```
-- Compact small files
OPTIMIZE catalog.sales.orders;

-- Remove old data files (default retention = 7 days / 168 hrs)
VACUUM catalog.sales.orders RETAIN 168 HOURS;
VACUUM catalog.sales.orders RETAIN 168 HOURS DRY RUN; -- preview only

-- To go below 7 days, disable safety check first:
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM catalog.sales.orders RETAIN 24 HOURS;

-- LIQUID CLUSTERING (modern replacement for ZORDER + static partitions)
-- Define at table creation:
CREATE OR REPLACE TABLE catalog.sales.orders
CLUSTER BY (customer, order_date)
AS SELECT * FROM staging.orders;

-- Change clustering columns on existing table (no full rewrite needed)
ALTER TABLE catalog.sales.orders CLUSTER BY (region, status);

-- Apply clustering – just OPTIMIZE, no ZORDER required
OPTIMIZE catalog.sales.orders;
```

Exam tip: Liquid Clustering vs ZORDER: ZORDER is per-OPTIMIZE, fixed columns. Liquid Clustering is defined on the table, works incrementally, columns changeable anytime. New tables -> prefer Liquid Clustering.

05 PySpark — read/write formats with explicit schema

PySpark

```
PySpark

from pyspark.sql.types import StructType, StructField, StringType, LongType, DoubleType

# Always define schema explicitly in production (not inferSchema)
schema = StructType([
    StructField('order_id', LongType(), nullable=False),
    StructField('customer', StringType(), nullable=True),
    StructField('amount', DoubleType(), nullable=True),
])

# Read JSON
df = spark.read.schema(schema).option('multiLine', True).json('/mnt/raw/orders.json')

# Read CSV
df = (spark.read
    .option('header', True).option('sep', ',')
    .schema(schema).csv('/mnt/raw/orders.csv'))

# Write modes: overwrite | append | ignore | errorIfExists
(df.write.format('delta')
    .mode('overwrite')
    .option('overwriteSchema', 'true')
    .saveAsTable('catalog.silver.orders'))
```

06 PySpark — column & row operations

PySpark

```
PySpark

from pyspark.sql.functions import (
    col, upper, to_date, when, coalesce
)

df = (df
    # Add / transform columns
    .withColumn('customer_upper', upper(col('customer')))
    .withColumn('order_date', to_date(col('order_date_str'), 'yyyy-MM-dd'))
    .withColumnRenamed('amount', 'order_amount')

    # Drop columns
    .drop('order_date_str', 'internal_id')

    # Filter rows
    .filter(col('order_amount') > 0)
    .filter(col('status').isNotNull())

    # Conditional column (equivalent to SQL CASE WHEN)
    .withColumn('tier',
        when(col('order_amount') > 1000, 'high')
        .when(col('order_amount') > 500, 'medium')
        .otherwise('low'))
)
```

07 PySpark — aggregations (count, sum, approx_count_distinct)

PySpark

```
PySpark

from pyspark.sql.functions import (
    count, count_distinct, approx_count_distinct,
    sum, avg, min, max
)

# Aggregation – this pattern appears in official exam sample question
daily_rev = (billing_df
    .groupBy('billing_date')
    .agg(
        sum('amount_billed').alias('total_revenue'),
        count_distinct('billing_id').alias('total_invoices'), # exact unique
        approx_count_distinct('patient_id').alias('est_patients'), # faster approx
        avg('amount_billed').alias('avg_amount')
    ))

# summary() – quick stats across all numeric columns
df.summary().show()
```

Exam tip: Exam Q1 tests exactly this: `count_distinct('billing_id')` for unique invoice count — NOT `sum('billing_id')`. Read carefully: 'unique invoices' = `count_distinct`, not `sum`.

08 PySpark — all join types + broadcast hint

PySpark

```
PySpark

from pyspark.sql.functions import broadcast

# INNER (default)
result = orders.join(customers, 'customer_id', 'inner')

# LEFT join – keep all orders even without customer match
result = orders.join(customers, 'customer_id', 'left')

# Multiple join keys
result = orders.join(
    customers,
    (orders.customer_id == customers.id) & (orders.region == customers.region),
    'inner'
)

# BROADCAST join – skip shuffle for small dimension table
result = orders.join(broadcast(customers), 'customer_id')

# UNION (deduplicated) vs UNION ALL (keep all rows)
combined = df1.union(df2)
combined_all = df1.unionAll(df2)

# CROSS join – cartesian product
cross = orders.crossJoin(regions)
```

09 PySpark — deduplication

PySpark

PySpark

```
# Remove fully duplicate rows
df_deduped = df.dropDuplicates()

# Deduplicate on subset of columns
df_deduped = df.dropDuplicates(['order_id', 'customer_id'])

# Keep MOST RECENT record per key (window-based dedup)
from pyspark.sql.functions import row_number, col
from pyspark.sql import Window

w = Window.partitionBy('order_id').orderBy(col('updated_at').desc())
df_latest = (df
    .withColumn('rn', row_number().over(w))
    .filter(col('rn') == 1)
    .drop('rn'))
```

10 PySpark — window functions (RANK, LAG, running total)

PySpark

PySpark

```
from pyspark.sql import Window
from pyspark.sql.functions import rank, dense_rank, lag, lead, sum as spark_sum

w_rank = Window.partitionBy('customer').orderBy(col('amount').desc())
w_time = Window.partitionBy('customer').orderBy('order_date')
w_run = w_time.rowsBetween(Window.unboundedPreceding, Window.currentRow)

result = (df
    .withColumn('rank', rank().over(w_rank))
    .withColumn('dense_rank', dense_rank().over(w_rank))
    .withColumn('prev_amount', lag('amount', 1).over(w_time))
    .withColumn('next_amount', lead('amount', 1).over(w_time))
    .withColumn('running_total', spark_sum('amount').over(w_run))
)
```

11 PySpark — UDFs (Python user-defined functions)

PySpark

PySpark

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# Define and register
@udf(returnType=StringType())
def categorize_amount(amount):
    if amount is None: return 'unknown'
    elif amount > 1000: return 'high'
    elif amount > 500: return 'medium'
    else: return 'low'

# Use in DataFrame
df.withColumn('tier', categorize_amount(col('amount')))

# Register for use in SQL
spark.udf.register('categorize_amount', categorize_amount)
spark.sql('SELECT categorize_amount(amount) FROM catalog.sales.orders')
```

Performance: Python UDFs serialize every row to Python and back — expensive. Prefer built-in Spark functions (when/otherwise). Use UDFs only for complex logic not available as built-ins.

12 Spark SQL — aggregations, HAVING, CASE WHEN

SQL

SQL

```
-- Aggregation with HAVING filter on groups
SELECT customer,
COUNT(*) AS order_cnt,
SUM(amount) AS total,
ROUND(AVG(amount), 2) AS avg_amount,
MIN(order_date) AS first_order
FROM catalog.sales.orders
WHERE status != 'cancelled'
GROUP BY customer
HAVING COUNT(*) > 5
ORDER BY total DESC;

-- CASE WHEN inline conditional
SELECT order_id,
CASE WHEN amount > 1000 THEN 'high'
WHEN amount > 500 THEN 'medium'
ELSE 'low'
END AS tier
FROM catalog.sales.orders;
```

13 Spark SQL — window functions

SQL

SQL

```
SELECT customer, order_date, amount,
RANK() OVER (PARTITION BY customer ORDER BY amount DESC) AS rnk,
DENSE_RANK() OVER (PARTITION BY customer ORDER BY amount DESC) AS dense_rnk,
ROW_NUMBER() OVER (PARTITION BY customer ORDER BY order_date) AS row_num,
LAG(amount, 1, 0) OVER (PARTITION BY customer ORDER BY order_date) AS prev_amt,
LEAD(amount,1, 0) OVER (PARTITION BY customer ORDER BY order_date) AS next_amt,
SUM(amount) OVER (
PARTITION BY customer ORDER BY order_date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS running_total
FROM catalog.sales.orders;
```


14 Spark SQL — higher-order functions on arrays

SQL

SQL

```
SELECT order_id,
-- TRANSFORM: apply lambda to every element
TRANSFORM(items, x -> x * 1.18) AS items_with_tax,
-- FILTER: keep elements matching condition
FILTER(items, x -> x > 100) AS expensive_items,
-- EXISTS: true if any element matches
EXISTS(items, x -> x > 500) AS has_big_item,
-- REDUCE: fold array to single value
REDUCE(items, 0D, (acc, x) -> acc + x) AS item_total
FROM orders_with_arrays;

-- EXPLODE: one row per array element
SELECT order_id, explode(items) AS item_price
FROM orders_with_arrays;

-- FLATTEN: merge nested arrays
SELECT flatten(array(array(1,2), array(3,4))) AS flat_arr;
```

15 Spark SQL — views (persistent, temp, materialized)

SQL

Both

```
-- Persistent view: stored in catalog, always re-executes query
CREATE OR REPLACE VIEW catalog.gold.high_value_orders
AS SELECT * FROM catalog.silver.orders WHERE amount > 1000;

-- Temp view: session-scoped only (gone on cluster restart)
CREATE OR REPLACE TEMP VIEW session_orders
AS SELECT * FROM catalog.silver.orders WHERE status = 'confirmed';

-- In PySpark: create temp view from DataFrame
df.createOrReplaceTempView('orders_temp')
spark.sql('SELECT * FROM orders_temp WHERE amount > 500')
```

Exam tip: View = always re-runs query (fresh data). Temp view = session only. Materialized view = pre-computed result refreshed by pipeline run. The exam asks which to use for BI performance vs freshness.

16 Lakeflow Spark Declarative Pipelines — 2026 API

PySpark

NEW

IMPORTANT: The dlt module is replaced by pyspark.pipelines. @dlt.table with readStream -> @dp.table. @dlt.table with batch DF -> @dp.materialized_view. @dlt.view -> @dp.temporary_view. Old code still works but new API is recommended.

PySpark — Lakeflow SDP 2026

```
# NEW import (replaces: import dlt)
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp

# BRONZE: @dp.table = streaming table (readStream source)
@dp.table(
    name = 'orders_bronze',
    comment = 'Raw orders from cloud storage'
)

def orders_bronze():
    return (spark.readStream
        .format('cloudFiles')
        .option('cloudFiles.format', 'json')
        .option('cloudFiles.schemaLocation', '/ckpt/schema')
        .load('/mnt/raw/orders/'))

# SILVER: streaming table with data quality expectations
@dp.table()
@dp.expect_or_drop('valid_amount', 'amount > 0') # drop bad row
@dp.expect('valid_customer', 'customer IS NOT NULL') # warn, keep row
@dp.expect_or_fail('valid_id', 'order_id IS NOT NULL') # halt pipeline
def orders_silver():
    return (spark.readStream
        .table('LIVE.orders_bronze')
        .withColumn('_load_ts', current_timestamp()))

# GOLD: @dp.materialized_view = batch/aggregated (not streaming)
@dp.materialized_view(comment='Daily revenue per customer')
def customer_daily_revenue():
    return (spark.read
        .table('LIVE.orders_silver')
        .groupBy('customer', 'order_date')
        .sum('amount'))

# Temporary view (replaces @dlt.view)
@dp.temporary_view()
def high_value():
    return spark.read.table('LIVE.orders_silver').filter(col('amount') > 1000)
```

17 Lakeflow SDP — CDC with create_auto_cdc_flow

PySpark

SQL

NEW

PySpark

```
# NEW API: dp.create_auto_cdc_flow (replaces dlt.apply_changes)
from pyspark import pipelines as dp
from pyspark.sql.functions import col

# First declare the target streaming table
@dp.table()
def orders_current():
    return dp.read_stream('orders_cdc_feed')

# Then define the CDC flow into it
dp.create_auto_cdc_flow(
    target = 'orders_current',
    source = 'orders_cdc_feed',
    keys = ['order_id'],
    sequence_by = col('updated_at'),
    apply_as_deletes = col('operation') == 'DELETE',
    except_column_list = ['operation', 'updated_at'] # exclude CDC meta cols
)
```

SQL

```
-- SQL equivalent (syntax unchanged from DLT)
CREATE OR REFRESH STREAMING TABLE orders_current;

APPLY CHANGES INTO LIVE.orders_current
FROM STREAM(LIVE.orders_cdc_feed)
KEYS (order_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY updated_at
COLUMNS * EXCEPT (operation, updated_at);
```

18 Spark tuning parameters (exam-required 4)

PySpark

PySpark / Config

```
# These 4 params are explicitly listed in the 2026 DEA exam syllabus

# 1. Shuffle partitions — default 200; tune DOWN for small data
spark.conf.set('spark.sql.shuffle.partitions', '32')

# 2. Default parallelism — controls RDD-based operations
spark.conf.set('spark.default.parallelism', '64')

# 3. Executor memory — set in cluster config to fix OOM
# spark.executor.memory = 8g

# 4. Broadcast join threshold — tables smaller than this auto-broadcast
spark.conf.set('spark.sql.autoBroadcastJoinThreshold', '52428800') # 50MB

# Delta write optimizations (reduce small file problem)
spark.conf.set('spark.databricks.delta.optimizeWrite.enabled', 'true')
spark.conf.set('spark.databricks.delta.autoCompact.enabled', 'true')
```

4

Lakeflow Jobs

~18%

01

Jobs / Tasks / Triggers — concepts & real limits

Concept

Concept	Description
Job	Primary resource for scheduling + orchestration. Has triggers, parameters, notifications, Git settings.
Task	Unit of work inside a job. Types: notebook, pipeline, python_script, sql, dashboard, dbt, if_else, for_each, spark_jar
Trigger	Initiates a job run: scheduled (cron), file arrival, table update, manual, continuous
DAG	Tasks visualized as Directed Acyclic Graph — dependencies define execution order
Repair Run	Re-runs only FAILED tasks and downstream; skips already-succeeded tasks

Limit	Value
Max concurrent task runs / workspace	2,000
Max jobs created per hour / workspace	10,000
Max saved jobs per workspace	12,000
Max tasks per job	1,000

02 DAB YAML — Lakeflow Job with branching & pipeline task

NEW

YAML — databricks.yml

```
# databricks.yml – complete job with if/else control flow
resources:
  jobs:
    etl_pipeline:
      name: 'ETL Pipeline with branching'
      trigger:
        schedule:
          quartz_cron_expression: '0 0 2 * * ?' # daily at 2 AM
          timezone_id: 'Asia/Kolkata'
      tasks:
        - task_key: ingest
      notebook_task:
        notebook_path: ./notebooks/01_ingest
        job_cluster_key: main_cluster
        - task_key: check_nulls
      depends_on:
        - task_key: ingest
      condition_task:
        left: '{{tasks.ingest.values.null_count}}'
        op: 'EQUAL_TO'
        right: '0'
        - task_key: transform
      depends_on:
        - task_key: check_nulls
      outcome: 'true' # runs only if no nulls
      pipeline_task:
        pipeline_id: '{{resources.pipelines.silver_pipeline.id}}'
        - task_key: fix_nulls
      depends_on:
        - task_key: check_nulls
      outcome: 'false' # runs only if nulls found
      notebook_task:
        notebook_path: ./notebooks/fix_nulls
      job_clusters:
        - job_cluster_key: main_cluster
      new_cluster:
        spark_version: '15.4.x-scala2.12'
        node_type_id: 'i3.xlarge'
        num_workers: 4
```

03 dbutils — widgets, notebook run, file system, secrets

PySpark

```
PySpark

# Widgets: parameterize notebooks
dbutils.widgets.text('env', 'dev', 'Environment')
dbutils.widgets.dropdown('mode', 'full', ['full', 'incremental'])
env = dbutils.widgets.get('env')
mode = dbutils.widgets.get('mode')

# Notebook chaining – run child notebook and get return value
result = dbutils.notebook.run(
    '/Shared/pipelines/transform',
    timeout_seconds = 3600,
    arguments = {'env': env, 'date': '2024-01-01'}
)

# Return value from the CHILD notebook:
dbutils.notebook.exit('SUCCESS')

# File system operations
dbutils.fs.ls('/mnt/data/')
dbutils.fs.cp('/mnt/src/file.csv', '/mnt/dst/file.csv')
dbutils.fs.rm('/mnt/data/old/', recurse=True)
dbutils.fs.mkdirs('/mnt/data/new_folder/')

# Secrets – never hardcode credentials
token = dbutils.secrets.get(scope='my-scope', key='api-token')
pwd = dbutils.secrets.get(scope='my-scope', key='db-pass')
```

5

CI/CD (NEW Section — 2026)

~12%

01

Databricks Repos & Git integration — CLI commands

NEW

Databricks CLI

```
# Pull latest changes into a workspace Repo
databricks repos update --repo-id 123456 --branch main

# List all repos
databricks repos list

# Create a new repo from a remote URL
databricks repos create \
--url https://github.com/org/repo.git \
--provider gitHub

# Get info about a specific repo
databricks repos get --repo-id 123456
```

UI workflow: Clone repo -> Create feature branch -> Edit notebooks -> Commit & Push -> Create PR in GitHub/GitLab -> Merge -> CI/CD deploys via DAB bundle deploy.

02 Declarative Automation Bundles — full bundle structure

NEW

YAML — databricks.yml

```
# databricks.yml – root bundle configuration
bundle:
  name: my_etl_pipeline
  variables:
  catalog:
    default: dev_catalog
    description: Unity Catalog name for this environment
    cluster_size:
      default: small
  targets:
  dev:
    mode: development # adds [dev username] prefix to job names
    default: true
    variables:
      catalog: dev_catalog
    workspace:
      host: https://adb-xxxx.azuredatabricks.net
  staging:
    mode: production
    variables:
      catalog: staging_catalog
    workspace:
      host: https://adb-yyyy.azuredatabricks.net
  prod:
    mode: production
    variables:
      catalog: prod_catalog
    workspace:
      host: https://adb-zzzz.azuredatabricks.net
  resources:
  jobs:
  etl_job:
    name: ETL Pipeline - ${var.catalog}
  tasks:
    - task_key: ingest
  notebook_task:
    notebook_path: ./notebooks/01_ingest
  pipelines:
  silver_pipeline:
    name: Silver Transformation Pipeline
    libraries:
      - notebook:
        path: ./pipelines/silver
        catalog: ${var.catalog}
```


6

Monitoring, Troubleshooting & Optimization

~8%

01 Spark UI — identifying bottlenecks

Concept

Symptom in Spark UI	Root Cause	Fix
One task takes 100x longer than others in a stage	Data skew — one partition is huge	Salt the skewed key, use AQE skew join hint
Large Shuffle Read/Write bytes in stage metrics	Excessive shuffle from wide transformations	Reduce shuffle.partitions, use broadcast join
Spill (Memory) or Spill (Disk) columns non-zero	Executor ran out of memory	Increase executor memory or reduce partition size
Many small tasks (1KB each) in stage	Too many partitions / small file problem	OPTIMIZE / coalesce / repartition
High GC time in executor tab	Too many objects in JVM heap	Increase memory, use Kryo serialization

02 Data skew — salting pattern

PySpark

PySpark

```
from pyspark.sql.functions import concat, lit, rand, floor, explode, array
salt_factor = 10

# Salt the large (skewed) table – add random salt to key
df_orders = (df_orders
.withColumn('salt', (floor(rand() * salt_factor)).cast('string'))
.withColumn('salted_key', concat(col('customer_id'), lit('_'), col('salt'))))

# Explode the small (dimension) table – replicate for each salt value
df_customers = (df_customers
.withColumn('salt', explode(array([lit(str(i)) for i in range(salt_factor)])))
.withColumn('salted_key', concat(col('customer_id'), lit('_'), col('salt'))))

# Join on salted key – data spread evenly across partitions
result = df_orders.join(df_customers, 'salted_key').drop('salt', 'salted_key')
```

03 Jobs monitoring — run history with Databricks SDK

PySpark

PySpark / SDK

```
from databricks.sdk import WorkspaceClient
w = WorkspaceClient()
# Get last 10 runs for a job
runs = w.jobs.list_runs(job_id=12345, limit=10)
for run in runs:
    print(run.run_id, run.state.result_state, run.execution_duration)
# Get current notebook task context (inside a running task)
ctx = dbutils.notebook.entry_point.getDbutils().notebook().getContext()
run_id = ctx.currentRunId().get()
job_id = ctx.tags().get('jobId').get()
```

7

Governance & Security — Unity Catalog

~7%

01

Managed vs External tables — Unity Catalog

SQL

SQL

```
-- Three-level namespace: catalog.schema.table
CREATE CATALOG IF NOT EXISTS prod;
CREATE SCHEMA IF NOT EXISTS prod.sales;

-- MANAGED table: Databricks controls storage location
CREATE TABLE prod.sales.orders (
  order_id BIGINT, customer STRING, amount DOUBLE
) USING DELTA;

-- DROP managed table: deletes BOTH metadata AND data files

-- EXTERNAL table: you control storage location
CREATE TABLE prod.sales.orders_ext (
  order_id BIGINT, customer STRING, amount DOUBLE
) USING DELTA
LOCATION 'abfss://container@storage.dfs.core.windows.net/orders';

-- DROP external table: deletes metadata ONLY – data files remain
```

Exam tip: MANAGED DROP = removes data. EXTERNAL DROP = removes metadata only. This is a very common exam trap question.

02 GRANT / REVOKE / DENY — full hierarchy

SQL

SQL

```
-- Step 1: Grant catalog access (MANDATORY first step)
GRANT USE CATALOG ON CATALOG prod TO `data_analysts`;

-- Step 2: Grant schema access
GRANT USE SCHEMA ON SCHEMA prod.sales TO `data_analysts`;

-- Step 3: Grant object-level access
GRANT SELECT ON TABLE prod.sales.orders TO `data_analysts`;
GRANT SELECT ON SCHEMA prod.sales TO `data_analysts`; -- all tables
GRANT MODIFY ON TABLE prod.sales.orders TO `data_engineers`;
GRANT ALL PRIVILEGES ON SCHEMA prod.sales TO `pipeline_svc`;

-- REVOKE
REVOKE SELECT ON TABLE prod.sales.orders FROM `data_analysts`;

-- DENY – explicitly blocks, overrides GRANT
DENY SELECT ON TABLE prod.sales.pii_data FROM `data_analysts`;

-- Inspect permissions
SHOW GRANTS ON TABLE prod.sales.orders;
SHOW GRANTS ON SCHEMA prod.sales;
```

Exam tip: Exam Q2: For read-only access to a schema, correct answer is GRANT SELECT ON SCHEMA. ALL PRIVILEGES gives write access too — wrong for read-only. Always grant USE CATALOG and USE SCHEMA first.

03 Row & Column security — dynamic views

SQL

SQL

```
-- Column masking: hide PII from non-admin users
CREATE OR REPLACE VIEW prod.sales.orders_masked AS
SELECT
  order_id,
  CASE
    WHEN is_account_group_member('pii_admins') THEN customer
    ELSE CONCAT(LEFT(customer, 2), '****')
  END AS customer,
  amount,
  status,
  order_date
FROM prod.sales.orders;

-- Row-level security: users see only their own region
CREATE OR REPLACE VIEW prod.sales.orders_regional AS
SELECT * FROM prod.sales.orders
WHERE region = current_user()
OR is_account_group_member('global_admins');
```

04 ABAC — row filter function (centralized policy)

SQL

NEW

SQL

```
-- 1. Create a reusable filter function
CREATE OR REPLACE FUNCTION prod.security.region_filter(region_col STRING)
RETURN is_account_group_member('global_admins')
OR region_col = current_user();

-- 2. Apply to table — enforced on ALL queries automatically
ALTER TABLE prod.sales.orders
SET ROW FILTER prod.security.region_filter ON (region);

-- Remove the row filter
ALTER TABLE prod.sales.orders DROP ROW FILTER;
```

ABAC vs dynamic views: ABAC row filters are defined once and enforced centrally on the table — no need to maintain a separate secured view. Policy applies automatically across all queries.

05 Delta Sharing — share setup & permissions

SQL

NEW

SQL

```
-- 1. Create a share object
CREATE SHARE IF NOT EXISTS sales_share;

-- 2. Add tables to the share
ALTER SHARE sales_share ADD TABLE prod.sales.orders;

-- 3. Create a recipient (external partner — no Databricks account needed)
CREATE RECIPIENT external_partner
COMMENT 'External analytics partner';

-- 4. Grant READ to recipient on the share
GRANT SELECT ON SHARE sales_share TO RECIPIENT external_partner;

-- Internal teams: grant directly via Unity Catalog
GRANT SELECT, MODIFY ON TABLE prod.sales.orders TO `internal_team`;

SHOW SHARES;

SHOW RECIPIENTS IN SHARE sales_share;
```

Feature	Databricks-to-Databricks	Databricks-to-External
Protocol	Unity Catalog grants	Delta Sharing open protocol
Recipient needs Databricks?	Yes	No (Spark, pandas, PowerBI)
Data copied?	No — read in place	No — read in place
Cross-cloud cost?	Yes — egress fees	Yes — egress fees

06 Lakehouse Federation — foreign catalog (query in place)

SQL

NEW

SQL

```
-- 1. Create a connection to the external system
CREATE CONNECTION my_postgres_conn
TYPE POSTGRESQL
OPTIONS (
  host 'mydb.postgres.database.azure.com',
  port '5432',
  user SECRET ('scope', 'pg-user'),
  password SECRET ('scope', 'pg-pass')
);

-- 2. Create a foreign catalog pointing to external DB
CREATE FOREIGN CATALOG my_postgres_catalog
USING CONNECTION my_postgres_conn
OPTIONS (database 'orders_db');

-- 3. Query external tables like local Unity Catalog tables
SELECT * FROM my_postgres_catalog.public.orders LIMIT 100;

-- Grant access to the foreign catalog
GRANT USE CATALOG ON CATALOG my_postgres_catalog TO `analysts`;
```

Exam tip: Lakehouse Federation = query external databases in place, no ingestion needed. Best for ad hoc queries. For heavy workloads, ingestion into Delta is still preferred.

Quick Reference — API Name Changes (DLT -> SDP 2026)

Area	Old (DLT)	New (SDP 2026)	Apache Spark?
Import	import dlt	from pyspark import pipelines as dp	Yes
Streaming table	@dlt.table + readStream	@dp.table	Yes
Materialized view	@dlt.table + batch DF	@dp.materialized_view	Yes
Temp view	@dlt.view	@dp.temporary_view	Yes
Append flow	@dlt.append_flow	@dp.append_flow	Yes
CDC / Apply Changes	dlt.apply_changes(...)	dp.create_auto_cdc_flow(...)	No
Expectations	@dlt.expect(...)	@dp.expect(...)	No
Sink	dlt.create_sink(...)	dp.create_sink(...)	Yes
Read upstream	dlt.read_stream('name')	spark.readStream.table('LIV E.name')	Yes
SQL streaming	CREATE STREAMING TABLE	CREATE STREAMING TABLE (unchanged)	Yes
SQL materialized	CREATE MATERIALIZED VIEW	CREATE MATERIALIZED VIEW (unchanged)	Yes

Ingestion Method Decision Guide

Source Type	Recommended Method
Files in S3/ADLS/GCS, continuous or large-scale	Auto Loader (cloudFiles)
Files in S3/ADLS/GCS, SQL-first, small-medium batches	COPY INTO
Salesforce, ServiceNow, Workday, HubSpot, Jira	Lakeflow Connect — SaaS connector
MySQL, PostgreSQL, SQL Server (incremental/CDC)	Lakeflow Connect — Database connector
Custom DB with full control, one-off	JDBC in notebook + Lakeflow Jobs
Kafka, EventHub, Kinesis, Pub/Sub streaming	Lakeflow Connect or Structured Streaming
External DB, query in place, no movement	Lakehouse Federation (foreign catalog)